

Clase 072 — Metasploit Framework: arquitectura y uso

Parte: **3 — Hacking ético y pentesting: metodología** · Fuente: *Kennedy et al. - "Metasploit: The Penetration Tester's Guide"*; documentación oficial de *Rapid7/Metasploit* 🕒 Duración estimada: **110 min**
· Nivel: **Intermedio**

Objetivo

El alumno comprenderá la arquitectura interna de Metasploit Framework —módulos, datastore, base de datos y workspaces— y adquirirá soltura operando `msfconsole`: buscar módulos, leer su documentación, configurar opciones y encadenar reconocimiento con explotación. Esta clase es la base técnica sobre la que se apoyan las tres clases siguientes del curso (explotación, Meterpreter y msfvenom).

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. Describir los seis tipos de módulos de Metasploit (exploit, payload, auxiliary, post, encoder, nop) y su función en la cadena de ataque.
2. Inicializar la base de datos de Metasploit y organizar el trabajo en workspaces separados por objetivo.
3. Buscar, seleccionar y configurar módulos usando `search`, `use`, `set`, `setg` y `options`.
4. Importar resultados de Nmap dentro de Metasploit para poblar hosts y servicios automáticamente.
5. Ejecutar módulos auxiliares de escaneo y enumeración sin necesidad de un exploit.
6. Explicar por qué separar el datastore global del local evita errores de configuración en engagements con múltiples objetivos.

Temas

#	Tema	Por qué importa
1	Arquitectura: exploit, payload, auxiliary, post, encoder, nop	Cada tipo de módulo cumple un rol distinto en la cadena de ataque
2	msfconsole y comandos base	Es la interfaz principal de trabajo del framework
3	Base de datos PostgreSQL y <code>msfdb</code>	Persiste hosts, servicios, credenciales y loot entre sesiones
4	Workspaces	Aísla los datos de distintos clientes o fases de un mismo engagement
5	Datastore: <code>set</code> vs <code>setg</code>	Evita reescribir RHOSTS/LHOST en cada módulo nuevo
6	Importación de Nmap (<code>db_nmap</code> , <code>db_import</code>)	Une la fase de reconocimiento con la de explotación sin trabajo manual
7	Módulos auxiliares	Permiten escaneo y enumeración detallada sin lanzar un exploit
8	Sesiones, jobs y background	Gestiona múltiples objetivos y handlers en paralelo

Definiciones y características

- **Exploit:** módulo que aprovecha una vulnerabilidad específica de un servicio o software. Característica clave: siempre se combina con un payload para lograr ejecución de código en el objetivo.
- **Payload:** código que se ejecuta tras una explotación exitosa (ej. shell de comandos, Meterpreter). Característica clave: puede ser staged (se envía en etapas) o stageless (se envía completo de una vez).
- **Auxiliary:** módulo sin payload asociado, usado para escaneo, fuzzing o fuerza bruta. Característica clave: es la herramienta principal para enumeración profunda dentro del propio framework.
- **Post:** módulo que se ejecuta sobre una sesión ya establecida. Característica clave: automatiza tareas de post-explotación como recolección de credenciales o información del sistema.
- **Datastore:** almacén interno de variables de configuración (RHOSTS, LHOST, LPORT, etc.). Característica clave: `setg` fija un valor global visible para todos los módulos, mientras `set` lo fija solo para el módulo activo.
- **Workspace:** contenedor lógico dentro de la base de datos de Metasploit que agrupa hosts, servicios y hallazgos de un mismo engagement. Característica clave: separa físicamente los datos de distintos clientes o VMs de laboratorio.
- **msfdb:** utilidad de inicialización y gestión de la base de datos PostgreSQL que respalda a Metasploit. Característica clave: sin ella, Metasploit funciona pero pierde persistencia entre sesiones.

Herramientas y preparación

- Kali Linux o Parrot OS con Metasploit Framework preinstalado (`msfconsole --version` para confirmar).
- PostgreSQL como backend de base de datos, gestionado por `msfdb`.

- Metasploitable2 desplegada en una red interna aislada (host-only) como objetivo de laboratorio.
- Nmap para el reconocimiento previo que luego se importará a Metasploit.

Inicialización recomendada:

```
sudo msfdb init
msfconsole -q
msf6 > db_status
# Debe responder algo como: "Connected to msf. Connection type: postgresql."
```

 **Nota ética:** Metasploit es una herramienta ofensiva real y potente. Todos los módulos que se usan en esta clase y en las tres siguientes se ejecutan **exclusivamente** contra máquinas de laboratorio propias (VMs propias, Metasploitable, rangos tipo HackTheBox/TryHackMe) o dentro de un alcance autorizado por escrito mediante contrato de pentesting con reglas de engagement (RoE) firmadas. El acceso no autorizado a sistemas de terceros es ilegal, independientemente de la intención o del resultado obtenido.

Laboratorio guiado

1. Arranca la consola y confirma el estado de la base de datos: `msfconsole -q` seguido de `db_status`.
2. Crea y selecciona un workspace dedicado al laboratorio: `workspace -a lab_parte3` y luego `workspace lab_parte3`.
3. Ejecuta un escaneo de Nmap directamente desde Metasploit para poblar la base de datos: `db_nmap -sV -p- 192.168.56.101`.
4. Revisa los datos importados con `hosts` y `services`; confirma que aparecen los puertos y versiones detectados.
5. Busca módulos relacionados con un servicio identificado, por ejemplo vsftpd: `search vsftpd`.
6. Selecciona el módulo correspondiente y revisa su documentación: `use exploit/unix/ftp/vsftpd_234_backdoor` seguido de `info`.
7. Revisa las opciones requeridas con `options` y fíjalas: `set RHOSTS 192.168.56.101`.
8. Fija el mismo valor como variable global para reutilizarlo en otros módulos: `setg RHOSTS 192.168.56.101`.
9. Ejecuta un módulo auxiliar de enumeración sin explotar nada: `use auxiliary/scanner/smb/smb_version`, `set RHOSTS 192.168.56.101`, `run`.
10. Revisa los datos acumulados en la base de datos del workspace: `hosts`, `services`, `vulns`, `notes`.
11. Crea un segundo workspace (`workspace -a lab_verificacion`) y repite un escaneo, confirmando que los datos no se mezclan entre workspaces.

Ejercicios

1. Explica la diferencia entre `set` y `setg` con un ejemplo concreto de RHOSTS y LHOST.
2. Enumera los seis tipos de módulos de Metasploit y da un caso de uso real de cada uno.
3. Importa un archivo XML de Nmap generado previamente con `db_import` y verifica los hosts cargados.
4. Usa un módulo auxiliar para detectar la versión de SSH del objetivo de laboratorio.
5. Crea dos workspaces distintos y demuestra, mostrando `hosts` en cada uno, que los datos no se mezclan.

6. Filtra módulos con `search` combinando `type:exploit` y `platform:unix` y explica qué representa cada filtro.

Reto verificable

Reto: Dejar un workspace de laboratorio completamente poblado: hosts descubiertos, servicios con versión importados desde Nmap, y al menos un módulo auxiliar ejecutado con su resultado documentado.

Criterio de aceptación: el comando `hosts` muestra el objetivo de laboratorio, `services` lista al menos tres servicios con su versión, y existe al menos una entrada generada por un módulo auxiliar visible en `notes` o `vulns`. Todo debe estar contenido dentro de un workspace nombrado explícitamente (no en `default`).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>db_status</code> responde "No connection"	PostgreSQL detenido o <code>msfdb</code> no inicializada; ejecutar <code>sudo systemctl start postgresql</code> y luego <code>msfdb init</code>
<code>search</code> devuelve cientos de resultados	Búsqueda sin filtros; usar <code>type:</code> , <code>platform:</code> o <code>cve:</code> para acotar
Las opciones no persisten al cambiar de módulo	Se usó <code>set</code> en lugar de <code>setg</code> para variables comunes como <code>RHOSTS/LHOST</code>
<code>db_nmap</code> corre pero no aparece nada en <code>hosts</code>	No hay workspace activo o la base de datos no está conectada; verificar con <code>db_status</code> y <code>workspace</code>
El módulo buscado no existe	Ruta escrita a mano con errores; copiar el path exacto que devuelve <code>search</code>
<code>msfconsole</code> tarda mucho en arrancar	Está reconstruyendo la caché de módulos tras una actualización; es normal la primera vez, esperar

Preguntas frecuentes

? ¿Necesito obligatoriamente la base de datos para usar Metasploit? No es obligatoria para lanzar un único exploit, pero es muy recomendable en cualquier engagement real: persiste hosts, servicios, credenciales y loot entre sesiones, algo esencial cuando hay múltiples objetivos.

? ¿Cuál es la diferencia real entre un módulo auxiliary y un exploit? Los módulos auxiliary no entregan un payload ni buscan ejecución de código; sirven para escaneo, enumeración o fuerza bruta. Los exploit sí buscan ejecutar código en el objetivo y siempre se combinan con un payload.

? ¿Metasploit Pro aporta mucho valor sobre el Framework gratuito? Pro añade automatización de flujos, generación de informes y una interfaz gráfica, pero el Framework gratuito cubre la enorme mayoría del contenido técnico necesario para aprender y para ejecutar engagements reales.

? ¿Puedo agregar mis propios módulos a Metasploit? Sí. El framework está escrito en Ruby y es completamente modular; los módulos personalizados pueden colocarse en `~/.msf4/modules` respetando la estructura de carpetas (`exploits/`, `auxiliary/`, etc.) del framework oficial.

? ¿Por qué usar workspaces si solo tengo una VM de laboratorio? Aunque el laboratorio tenga un solo objetivo, practicar con workspaces prepara al alumno para engagements reales donde se manejan decenas de clientes y es fácil confundir datos si todo vive en el workspace `default`.

Referencias

- Kennedy, D. et al. "Metasploit: The Penetration Tester's Guide" (No Starch Press) — referencia histórica de arquitectura del framework.
- Rapid7, documentación oficial de Metasploit: <https://docs.metasploit.com/>
- Offensive Security, Metasploit Unleashed: <https://www.offsec.com/metasploit-unleashed/>
- PTES (Penetration Testing Execution Standard), fase de Exploitation: <http://www.pentest-standard.org>
- MITRE ATT&CK, técnica T1595 (Active Scanning) como contexto de la fase previa a la explotación: <https://attack.mitre.org/techniques/T1595/>

Siguiendo clase

Clase 073 - Metasploit: explotacion y payloads